

Precision meets speed through an FPGA-based natively sparse Ising machine for combinatorial optimization

Received: 23 September 2025

Accepted: 23 June 2026

Published online: 11 July 2026

Cite this article as: Yao, B., Ebler, D., Shi, X. *et al.* Precision meets speed through an FPGA-based natively sparse Ising machine for combinatorial optimization. *Nat Commun* (2026). <https://doi.org/10.1038/s41467-026-75119-0>

Baijian Yao, Daniel Ebler, Xu Shi, Juntao Wang, Fan Zhang, Wei Zhang & Jie Sun

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

Precision meets speed through an FPGA-based natively-sparse Ising machine for combinatorial optimization

Baijian Yao¹, Daniel Ebler ^{*2}, Xu Shi¹, Juntao Wang², Fan Zhang², Wei Zhang ^{†1}, and Jie Sun ^{‡2}

¹Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong SAR, China

²Leibniz Theory Lab, 2012 Labs, Huawei Technologies Co. Ltd., Hong Kong SAR, China.

Abstract

Ising machines constitute a framework to solve combinatorial optimization problems efficiently utilizing dynamical interacting systems. Typical approaches simulate the dynamics on digital hardware, which is strongly dependent on matrix-vector operations, and were shown to experience acceleration of several orders of magnitude by parallel processing on FPGAs. However, optimization on single FPGAs has remained limited to small-scale problems due to memory limitations, and scaling on multi-FPGA networks is challenging to operate. In this work, we propose a high-accuracy and FPGA-based Ising machine design that achieves state-of-the-art solution speeds on integrated hardware and is scalable to problems of up to 20000 variables on a single unit. Concretely, we introduce a sparse data format, called tiled coordinate list, for more efficient memory use and accelerating matrix-vector operations. Further, we propose a quantization mechanism to 8-bit integer precision that allows to scale problem sizes without reducing the accuracy of the solution. Co-optimization of the data flow and hardware design strongly accelerates the Ising machine while reducing computational costs. This enables 10-fold up to 68-fold speedups on Max-Cut problems from the Gset graph collection compared to previous state-of-the-art FPGA-based Ising machines.

Introduction

For many disciplines, identifying accurate solutions to discrete optimization problems in the shortest possible time is of great importance, including wireless and optical communication networks, synthetic biology, power grids, high-performance computing, machine learning, logistics, and financial markets, among many others^{1–6}. However, given the hardness of the problems, the solution speed and accuracy of the solutions are often limited, especially because the algorithmic design of the solvers is usually decoupled from the supporting hardware architecture.

Recent approaches suggest Ising machines as a class of analog solvers for combinatorial optimization^{7–14}. The optimization problem is encoded into the interaction pattern of coupled dynamical systems, and temporal evolution, combined with external control mechanisms, drives the systems towards stable modes corresponding to candidate solutions. Compared to conventional solvers, state updates in Ising machines are intrinsically parallel. This has led to solution speeds several orders of magnitude faster than conventional solvers, for instance in physical networks such as coupled photons in fiber cavities, spintronics, and mechanical oscillators^{9 15 16}. However, scaling experiments to practical problem sizes is challenging and the accuracy of the solution varies strongly.

*Corresponding author: Mail address: ebler.dan@gmail.com

†Corresponding author: Mail address: wei.zhang@ust.hk

‡Corresponding author: Mail address: riosun@gmail.com

Simulation of the network dynamics on digital hardware mitigates experimental challenges while preserving the hardware acceleration of the solver^{10,13,17–19}. The dynamics is described by a set of coupled differential equations, which constitute an update rule consisting mainly of matrix-vector operations. This has led to multi-threaded solutions on Field Programmable Gate Arrays (FPGA), which achieve 2-3 orders of magnitude acceleration over conventional solvers²⁰. However, previous proposals were strongly limited in problem scale, mainly for two reasons. Firstly, the optimization problem was stored in dense matrix format²⁰, while many real-world problems are inherently sparse. Sparse FPGA implementations^{21,22} were proposed to address this issue, but were 2-3 orders of magnitude slower compared to dense implementations²⁰. Secondly, the matrices and vectors were stored within the FPGA chip memory, which is limited in capacity. To overcome these issues, multi-FPGA networks were proposed to scale problems to 100,000 variables²³. This setup, however, requires ultra-large bandwidth and cumbersome multi-chip synchronization, which is hard to control and replicate in practice.

In this work, we present an FPGA-based sparse Ising machine design that achieves state-of-the-art solution speed on digital hardware and stably high-accuracy solutions. The design is based on several insights. Firstly, we propose a new data format, the tiled coordinate list (TCOO), which allows matrices to be efficiently stored in as sparse matrices. Data are stored on off-chip FPGA memory, allowing for simultaneous computation and transmission, yielding on average 30% reduced memory usage compared to COO and 22% reduced memory usage compared to block COO (BCOO)²⁴. The format also enables better workload balancing between multiple processing elements (PEs), achieving on average a utilization rate of PE runtime relative to total system runtime of 95%. Secondly, we introduce a quantization method as part of a computing framework that converts data from double precision to Int8 format. This reduces the memory usage of the simulation of system dynamics by half and strongly reduces computational complexity. At the same time, the solution accuracy is maintained at 99% compared to double precision implementations. Lastly, built on these findings, we propose an implementation of quantized simulated bifurcation machines (qSBM) as an Ising machine on FPGA, which is co-optimized between data flow and hardware. The co-optimization improves both the hardware design of the FPGA and accelerates sparse matrix-vector (SpMV) operation. We demonstrate 10-fold to 68-fold (average 39-fold) speedups compared to previous state-of-the-art sparse FPGA implementations of Ising machines²¹. This sparse-to-sparse comparison highlights the architectural advantages of our approach.

Results

Combinatorial Optimization and Ising Machines. Combinatorial optimization problems of the quadratic type can be formulated as minimizing the energy of Ising Hamiltonians⁷

$$\min_{\sigma \in \{-1, +1\}^N} H_{\text{Ising}}(\sigma) = \min_{\sigma \in \{-1, +1\}^N} -\sigma^T J \sigma, \quad (1)$$

where σ contains the binary values of N spins coupled through the (generally symmetric) problem matrix J . Configurations σ^* that minimize energy are called ground states.

Ising machines encode Ising Hamiltonians into the interaction pattern of continuous dynamical systems in a way that yields the ground state when evolving the systems towards specific stable configurations. The general equations of the systems can be written as coupled ordinary differential equations (ODEs) as^{13,25}

$$\frac{d\mathbf{x}}{dt} = F(\mathbf{x}(t), \mathbf{p}(t)) + \xi \nabla H_{\text{Ising}}(\mathbf{x})(t) \quad (2)$$

where $\mathbf{x}$ corresponds to a continuous real-valued relaxation of the binary spins σ , and F denotes the motion of the individual particles which can be controlled by adjusting external parameters $\mathbf{p}(t)$ over time t . The coupling is scaled by a parameter ξ .

Simulations of Ising machine dynamics on digital hardware typically uses first-order methods to identify the

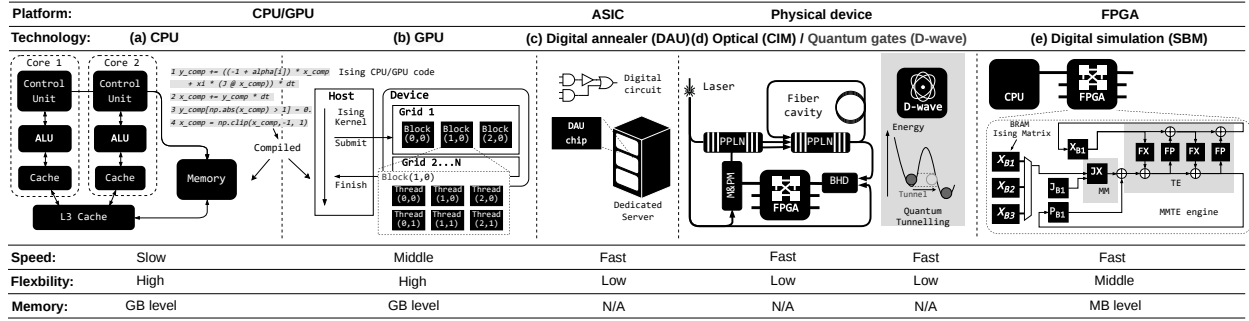


Figure 1: FPGA-based Ising machines in comparison with other Ising machine architectures. (a) CPU-based simulation, where compiled assembly code is stored in memory and executed on two cores. (b) GPU-based simulation using CUDA code compiled into machine instructions, dispatched by the host CPU, and parallelized across thousands of threads. (c) DAU, where parallel runs of simulated annealing are performed via a dedicated digital chip, assisted by a server. Physical devices: (d) CIM, an optical coherent Ising machine built on an optical fiber network, where the FPGA acts as a data transmitter and matrix multiplier; D-Wave, a quantum hardware-based Ising solver. (e) SBM, a digital FPGA-based simulator performing Ising simulation via a two-step process: matrix-vector multiplication (MM) and time evolution (TE).

fixed points of Eq. (2), which gives in the case of Euler method the following simple update rule

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t \cdot [F(\mathbf{x}(t), \mathbf{p}(t)) + \xi J\mathbf{x}] . \quad (3)$$

Notably, the main computational cost comes from matrix-vector multiplication $J\mathbf{x}$, while $F(\mathbf{x}(t), \mathbf{p}(t))$ consists of cheap vector-element-wise operations. We term operations other than $J\mathbf{x}$ as time evolution (TE). The binary value of the optimization variables is typically retrieved at the end of the evolution via $\sigma = \text{sign}(\mathbf{x})$.

Hardware implementations of Ising Machines. Implementations of Ising machines have been explored on several hardware platforms. For practical usage, it is important to achieve high performance in terms of accuracy and execution speed, input flexibility, and scalability.

General-purpose CPUs provide maximum flexibility, but are inefficient for fine-grained parallel-spin simulations, see Fig. 1(a). Here, the Ising simulation has to be compiled into assembly code and executed by only a few cores (two in the example). GPUs offer substantial parallelism for spin updates by providing thousands of execution threads and have been used to accelerate Ising simulations²⁶, yet their performance remains limited by memory hierarchy constraints, as the data moves between the host device and the GPU very often, see Fig. 1(b).

Dedicated Application-Specific Integrated Circuits (ASICs), such as Fujitsu's Digital Annealer²⁷ (see Fig. 1(c)) deliver the highest performance and energy efficiency through custom-designed threading of solver executions. However, the algorithm departs from the fine-grained parallel-spin simulation and implements a replica variant of simulated annealing, which is a discrete optimization algorithm differing strongly in convergence properties and parallelizability. In addition, the Digital Annealing Unit (DAU) can run only small- to medium-scale problems with a few thousand variables.

Emerging paradigms such as quantum annealing²⁸ are implemented in a superconducting hardware architecture, which can reach solutions more efficiently in theory through quantum tunneling (Fig. 1(d)). However, run-time limitations strongly limit machine accuracy, and scalability has been considered very challenging^{29,30}. Photonic implementations of Coherent Ising machines⁹ realize a combination of optical cavities and FPGAs (Fig. 1(d)), where a fiber cable guides the pulses and the FPGA serves as a data channel and

matrix vector multiplier. However, the design struggles with variable inputs and solution accuracy due to noise.

FPGA-based simulators can be configured to implement a custom spatial architecture with parallel MMTE engines, see Fig. 1(e). This approach offers near-ASIC efficiency while maintaining reconfigurability, making FPGA a highly flexible platform. Furthermore, the high multithreading capabilities of the FPGA-based architectures enable simultaneous updates of thousands of spins with minimal communication overhead²⁰, avoiding the memory bottlenecks of conventional architectures such as CPU and GPU. The primary constraint is on-chip memory capacity. The number of spins that can be simulated is limited by the available block RAM (BRAM) needed to store interaction coefficients (J_{ij}) and local fields (h_i). As problem sizes increase, storing the complete interaction matrix becomes challenging, necessitating efficient external memory access strategies. This work addresses this limitation through a novel architecture that optimizes data flow and memory hierarchy for large-scale sparse Ising problems, pushing the boundaries of reconfigurable computing while maintaining flexibility and precision.

TCOO: high efficiency data storage. The computationally most complex operation in the simulation of the Ising machine dynamics is the matrix-vector multiplication $J\mathbf{x}$ in Eq. 3. In the following, we introduce a new sparse data storage format, called the tiled coordinate list (TCOO), which stores the matrix J more efficiently and allows significant computational speed-ups. For a more detailed overview of the most common storage formats used in the following, we refer to Supplementary Note 2.

The data format in this work is based on the coordinate list (COO) format, introduced in³¹. The idea is to compress non-zero (NNZ) matrix elements into an array (*col, row, value*), denoting the column and row position of the element in the matrix, as well as its value (see Supplementary Figure 2). We observe that the overhead of addressing the coordinates of an element will increase with the matrix size and eventually form a computational bottleneck. To resolve this issue, we partition J into *tiles* of size $M \times M$, where M is an adjustable parameter. Fig. 2 (a) shows an example of a tiling for $M = 3$ where J is partitioned into four smaller tiles a, b, c , and an empty tile.

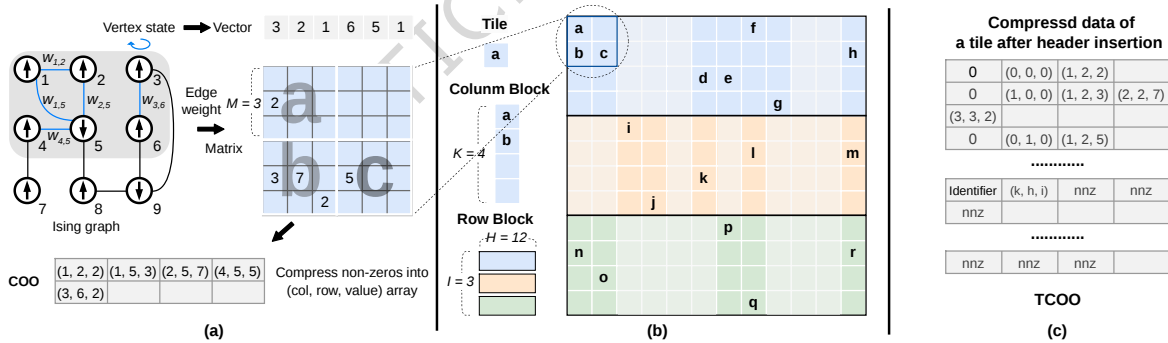


Figure 2: From Ising graph to TCOO: (a) The optimization problem is encoded in an Ising problem with interaction graph shown on the left. The adjacency matrix J of the graph is partitioned into tiles a, b, c of size $M = 3$ (and an empty tile), and stored in an array in COO format; (b) Tiles which are multiplied with the same vector segments are arranged in the same column blocks (the length of the blocks is $K = 4$ here). The column blocks are arranged in row blocks (indicated by the different colors) of length H (here, $H = 12$); (c) TCOO. the new data format stores values in arrays in the following order: Identifier (8-bit zero), header (coordinate of the tile), and non-zeros (coordinate inside the tile).

During matrix-vector multiplication, tiles within the same column are multiplied by the same vector segment. Hence, contiguous tiles in the same column are grouped as a column block of length K . When a processing element (PE) multiplies a tile with the respective vector segment, it loads the segment into its local memory.

To avoid repetitive loading of the vector segment, a larger column block length K is favorable, but generally K is limited by the local result memory size of the PE. Fig. 2 (b) shows one example with $K = 4$.

The column blocks are further arranged in row blocks of length H , see Fig. 2(b) where $H = 12$. There are in total $I = 3$ row blocks in the example, indicated by different colors. In sparse matrix-vector multiplication (SpMV), parts of the resulting output vector are computed earlier and can be readily used for further processing. For Ising machines, this allows us to simulate parts of the dynamics already while the other parts of SpMV are being computed. The length of the row block H determines the size of the parts.

During SpMV, the PE needs the coordinates of a tile to load it from the off-chip memory and to multiply it with the respective vector segment. The coordinates are also called the *header* and are received by the PE at the beginning of a calculation. For a tiled matrix with column block length K , row block length H and total number of row blocks I , the header of the tiles is a tuple (k, h, i) , with $k = 0, 1, \dots, K - 1$, $h = 0, 1, \dots, H - 1$, and $i = 0, 1, \dots, I - 1$. For example, in Fig. 2(b) tile d has the coordinates $(2, 5, 0)$.

The compressed data in TCOO then consist of an array with several different elements, see Fig. 2 (c). Firstly, an 8-bit zero $\mathbf{0}$ is prepended to the header as an identifier, telling the PE during processing to treat the header as the tile address instead of processing it as a value. We choose an 8-bit zero to ensure that after insertion the data boundaries will not shift, keeping the data aligned to the data bus width of the hardware, see Supplementary Figure 7. The identifier and header are followed by the values in the tiles. Each value is a NNZ with tuple $(col, row, value)$ indicating the position of the value inside the tile. Note that this differs from the COO format in Fig. 2 (a). If we consider for illustration purposes the single value with COO coordinate $(1, 5, 3)$, the TCOO format reads $[\mathbf{0}, (0, 1, 0), (1, 2, 3)]$.

We conclude by commenting on implementation efficiency trade-offs depending on the parameters in the TCOO design. Small values of the tile size M increase the number of headers, which leads to higher bandwidth consumption to transmit headers instead of NNZ, lowering the performance. Large values of M , on the other hand, reduce the utilization of the PEs due to decreased parallel processing. Small values K of the column block height lead to frequent vector loading, which causes PEs to stall due to memory contention. Large K , on the other hand, reduces the parallelism of the PE processing and increases the consumption of hardware resources. The number of row blocks I and the length H determine the execution performance of SpMV as short rows increase the loading frequency of the vector in the PE, while long rows decrease the parallelization in processing. To systematically explore these trade-offs, our implementation fully parametrizes both the TCOO format and hardware architecture, see Supplementary Note 3. This enables hardware-software co-optimization through exhaustive design space exploration. The final parameter selections are detailed in the Supplementary Note 6.

We benchmarked TCOO on random graphs from the Gset graph collection³² and found that TCOO achieves, on average, a 30.49% higher compression ratio compared to COO, and 22.11% compared to improved bitmap coordinate formats such as BCOO. (See Supplementary Table 3 for detailed data). In the meanwhile, we observe that in specific cases, CSR5³³ outperforms TCOO. The detailed discussion is given in Supplementary Note 4. This performance difference stems from the inherent structural limitations of simple tiling when applied to matrices with highly irregular, banded sparsity patterns—specifically. In case the tiles are perfectly aligned with dense bands, TCOO performs well, but for matrices with intersecting band edges that contain few non-zero elements, the storage efficiency is reduced as the fixed header overhead dominates.

Data Flow. The compressed TCOO data array of each tile is stored in off-chip memory (DRAM). To balance the workload of the PEs, a data queue for the tiles is scheduled in DRAM by a bin-packing algorithm that considers the vacancy of the PEs (denoted in a working queue) and the number of non-zeros in the column blocks, as described in the following. The column blocks will be sorted by the amount of NNZ. Then, the column block with the most NNZs will be scheduled to the most vacant PE until all column blocks have been scheduled. The column blocks will be written into DRAM in the same order as the scheduling.

A Dispatcher assigns the queued block from the DRAM and the respective vector segments to the PEs. The

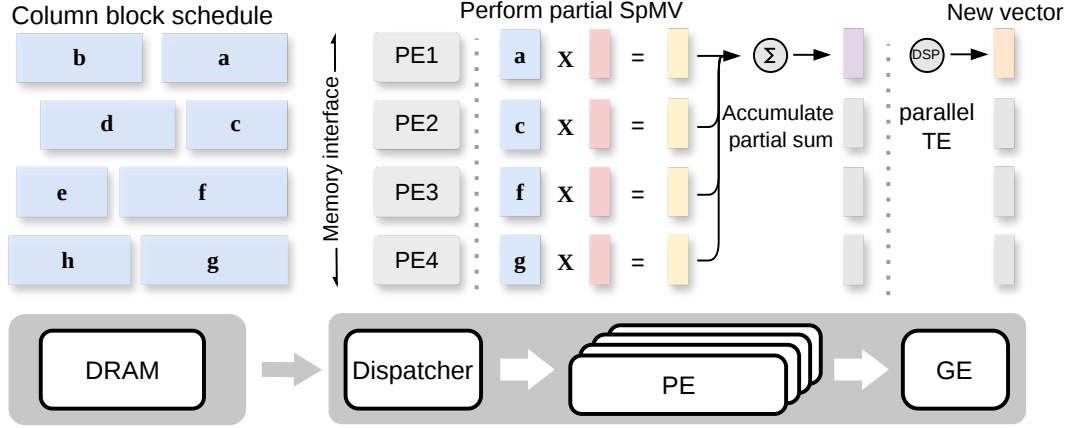


Figure 3: Data flow and architecture overview. The Dispatcher loads the tiles from DRAM and put them with the respective vector segments on the PEs, which perform partial SpMV in parallel. Once all column blocks in one row block are processed, the PEs add up the local SpMV results and send them to the gather element (GE). The GE uses the results to simulate parts of the Ising dynamics. Then, the Dispatcher moves to the next row block.

vector segments are of the same dimension M as the tiles and indexed by the same label h of the header (k, h, i) of the tiles. Then, each PE executes the matrix vector operation, stores the output in local memory, and receives the next unassigned block and vector segment from the dispatcher. The processing on the PEs is done fully in parallel. Fig. 3 shows one simple example in which each PE is scheduled with two tiles.

Before moving on to the next row block, all tiles in the current row block are processed such that the PEs are vacant again. This is necessary as each PE stores the outputs of the local multiplications, and because multiple PEs process the same row block, each of the PEs will contain part of the result. This leads to the partial sum issue identified in ³⁴, and is overcome by an accumulation element that performs a simple addition of parts once a row block is fully processed. The resulting vector segment is sent to the gather element (GE) which computes the time evolution (TE) of the system corresponding to the segment. The dispatcher then moves on to the next row block. After all row blocks are processed, the combination of all resulting vector segments carries the full result of one TE step of the Ising simulation.

Supplementary Figure 6 illustrates the full example, including the summation, in more detail.

Data Quantization. We introduce quantization ^{35–38} to improve the efficiency of storage and computation. Here, the goal is to convert a range of continuous values for the system variables into a finite range of discrete values, in our scenario Int8, while maintaining the accuracy of the Ising machine optimization. Int8 allows eight bits for the encoding and only represents integers.

The quantization maps a real value r to a discrete integer q as

$$q = \lceil r/S + Z \rceil, \quad S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} \quad (4)$$

where Z is the zero point, and S denotes the scaling factor defined by the maximum ranges $r_{\max} - r_{\min}$ of real numbers and discrete values $q_{\max} - q_{\min}$. The rounding operation maps the value to the next larger number in the discrete set of possible values for q . Adjusting Z compensates for the misalignment of the real numbers and the range of quantized numbers, which potentially leads to a loss of accuracy. For example, if $r_{\min} = 0$ but $q_{\min} = -128$ direct scaling would map $r = 0$ to $q = 0$, such that the quantized values do not

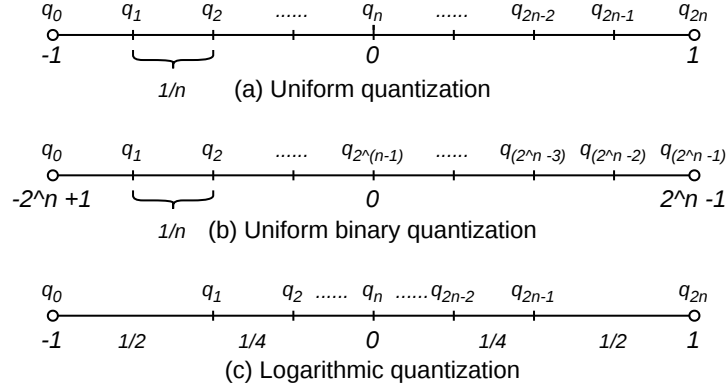


Figure 4: Quantization schemes: (a) Uniform quantization; (b) Uniform binary quantization; (c) Logarithmic quantization.

cover the original distribution. Adjusting $\mathbf{Z}$ (e.g. $\mathbf{Z} = -q_{\min}$) shifts the range such that $r_{\min}$ maps to $q_{\min}$ and $r_{\max}$ maps to $q_{\max}$, fully utilizing the available bins.

Previous work³⁹ has evaluated the performance of Ising machines under different quantization schemes and found good results with uniform quantization, while other schemes, such as logarithmic quantization, exhibited instabilities (see Fig. 4). We proposed a uniform binary quantization in the following that scales the original value range for the system states $x_i \in [-1, 1], i = 1, 2, \dots, N$, to integers $x_i \in \{-2^{n+1}, -2^n, \dots, 2^{n-1}\}$ (see Fig. 4), since integer multiplication is more compatible with the FPGA lookup table (LUT), see Supplementary Note 5.

In Ising machine simulations (see Eq. 3), the initial state $\mathbf{x}(t = 0)$ of the system is picked at random with entries close to zero. In uniform binary quantization, the state is scaled accordingly by choosing the closest discrete values. It was shown that such adjustments to initial state values can improve performance for random graph instances⁴⁰. In addition, system parameters such as $\mathbf{p}(t)$ and coupling ξ are also quantized accordingly. Note that the behavior of the Ising machine usually relies on a specific temporal schedule for $\mathbf{p}(t)$. Quantization changes such a schedule, which can disturb the convergence properties. For that reason, we used smaller step sizes Δt based on experimental results.

Quantization reduces the vector BRAM consumption of FPGAs by approximately 50%. In the implementation shown in Supplementary Table 5, this saves 48 BRAM tiles (15% of total FPGA BRAM resource).

Quantized Ising machines. We benchmark the performance of Ising machines with binary uniform quantization against full-precision floating-point implementations. We chose ballistic simulated bifurcation (bSB) as it has been shown to achieve stably high accuracy on different benchmarks⁴⁰. The dynamics of bSB executes additional thresholding mechanisms for error mitigation purposes (Supplementary Note 1). The quantized bSB algorithm, which we refer to as qSB in the following, is explained in detail in Supplementary Note 1.

For benchmarking, we choose random instances of the Max-cut problem (see Methods) from the Gset graph collection³², which has become a standard testbed for Ising machine performance assessment⁴¹. The selected problem sizes range from 800 to 4000 nodes. For each instance, ten separate runs are conducted, each with 1000 iterative steps. As shown in Fig. 5, in more than half of the instances, qSB has better solution accuracy than full-precision bSB, such as for G19, G21, G47 and G31, and matches the accuracy of bSB in the other instances. For the remaining instances, the accuracy is within 99% of the full precision floating point implementations. The Gset benchmark offers a controlled but simplified testbed, with weights $\{-1, 0, +1\}$.

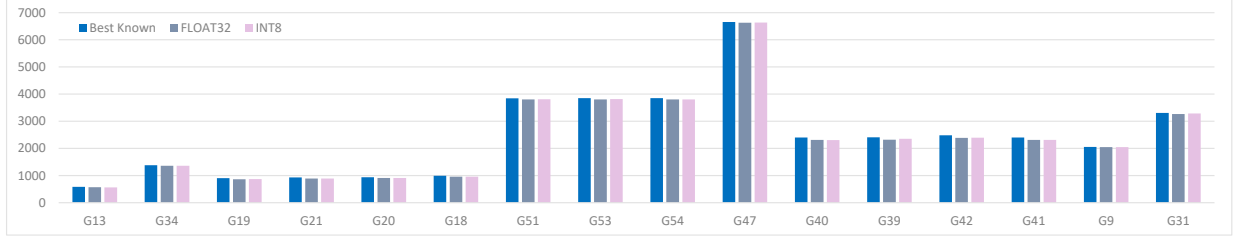


Figure 5: Average Max-cut values on Gset graphs. The figure compares the best known value to FLOAT32 (full precision) bSB runs and Int8 qSB runs.

To ensure that our quantization advantages generalize, we provide additional results in Supplementary Note 5, Figure 12 on graphs with diverse weight distributions, as well as performance benchmarking on more general QUBO problems: balanced minimal cut (bmincut), and QPLIB problems⁴². Which include constrained problems with a wide numerical range of couplings, structural heterogeneity, and mixed sparsity. These tests corroborate that the performance and stability of qSB are consistent across broader and more complex problem instances.

To understand the impact of quantization and parameter tuning on the performance of qSB, we perform a controlled ablation study in Supplementary Note 5. We find that quantization alone worsens the performance, as reduced precision induces larger local errors per step and coarsens the state-update granularity which impacts the solver’s ability to navigate the energy landscape. Consequently, applying INT8 quantization without any additional adjustments (Config B) leads to a notable increase in variance on certain instances. However, re-tuning the system parameters—most notably of the time step Δt —not only recovers the performance but allows to even surpass the FP32 baseline, see Config C. Concretely, the quantized system is found to achieve better accuracy, a lower average deviation and significantly lower variance. Importantly, this recovery is not achieved by trading computational efficiency for accuracy, as all experiments were conducted with an identical number of total steps. The inherent nonlinearities introduced by quantization—such as dynamic clipping and variable binarization—naturally align with the intrinsic nonlinear processes of the bSB algorithm. In summary, quantization alone does not offer a drop-in benefit; it must be paired with appropriate parameter re-tuning to realize its full potential. In practice, this re-tuning is a one-time offline step per problem class and does not compromise runtime efficiency, enabling the hardware gains (e.g., reduced memory footprint and integer-arithmetic acceleration) without sacrificing solution quality.

Co-optimized Ising machines on FPGA. Previous implementations of Ising machines in FPGAs suffered from several shortcomings. On dense FPGA architectures²⁰, the size of the optimization problems was limited to small-scale problems of around 4000 variables. In sparse FPGA architectures²¹, scalability was demonstrated to problems with up to 200000 variables, but the runtime was 2-3 orders of magnitude slower than in dense architectures. In the following, we propose a sparse hardware-software co-optimized FPGA design that realizes the scalability of previous sparse approaches while at the same time achieving state-of-the-art solution speed.

The design, which we call qSBM, is based on co-optimizing the FPGA architecture with the data format TCOO, the algorithm qSB which utilizes Int8 precision, and the corresponding data flow. We implement qSBM on Xilinx FPGAs ZCU106 utilizing SpinalHDL, a high-level hardware description language. In comparison, the state-of-the-art dense design **K2000**²⁰ is implemented on Intel FPGA Arria 10 GX 1150 using Verilog, and the state-of-the-art sparse design **MA**²¹ is implemented on Xilinx FPGAs ZCU106 using Vivado HLS. The complete implementation and hardware design are explained in the Supplementary Note 3 and Supplementary Table 2. We focus here mainly on the benchmarking of qSBM against the two designs mentioned above.

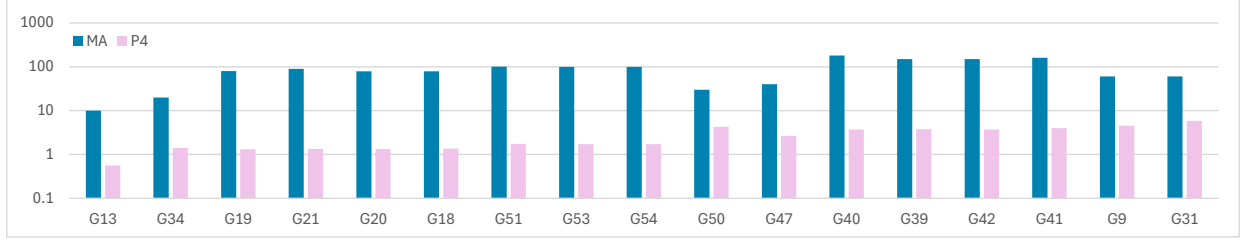


Figure 6: Time-to-target (ms) performance comparison for Gset on platform **MA**, and **P4** (1000 bSB/qSB steps).

The performance benchmark results are summarized in Fig. 6. We compare our qSBM design **P4**, which has 4 PEs, against the state-of-the-art sparse FPGA SBM **MA**²¹. The time to target is defined as the duration required to achieve 99% of the best known solution for an optimization task, which typically involves 1000 steps in bSB. The TCOO parameters M and K were decided prior to implementation and set to $M = 64$ and $K = 8$ (Supplementary Note 6). It can be seen that our FPGA implementation is 68x faster in the best case (G21) and 10x faster in the worst case (G31) than reported in reference²¹.

To evaluate the performance on large-scale Ising problems, we generated three sparse graph sets (A1, A2, A3), containing graphs of sizes 10,000, 16000 and 20,000 nodes. They are constructed to match the number of nonzero elements of the reference test cases²¹ C1, C2, and C3 (containing 524,000, 1,048,000, and 786,000 nonzero elements, respectively) used by **MA**. All graphs have density $< 1\%$. Our simulations demonstrated consistent average speedups of 22.3x, 22.5x, and 18x across the sets compared to the sparse design **MA**, with net execution times 112.2ms, 222.2ms, and 166.8ms. In particular, the distribution of graph elements, whether randomly distributed or tile-centered, did not affect the observed speedups, confirming the robustness of our design and the direct architectural improvements over previous approaches.

We also provide a fair comparison of architectural efficiency against the dense design **K2000**. We focus our analysis on G-set instances where the dense baseline **K2000** is not hindered by algorithmic convergence issues (see Supplementary Note 4 for methodological details). Among the 45 such instances, our sparse **P4** design achieves a lower time-per-step in 25 cases, with an average speedup of 5.1x. This includes significant speedups on challenging instances (e.g. $\sim 38x$ on G14, $\sim 71x$ on G35, $\sim 135x$ on G37). This comparison underscores the fundamental efficiency of our sparse architecture: its execution time scales with the number of non-zero entries, whereas a dense solver's time scales with N^2 .

To understand the sources of the performance advantages, we conducted a systematic ablation study to explore the individual impact of the tiled format, bin-packing-based load balancing, and hardware hazard handling within our TCOO architecture. As detailed in Supplementary Note 6, each component contributes significantly. Concretely, we find that bin-packing-based load balancing alone yields an average speedup of 1.2x. Independently, hardware hazard handling provides a more substantial average speedup of 2.5x by eliminating 59% of the graph preprocessing overheads. Most notably, the full integration of these techniques achieves a combined speedup of approximately 3x over the baseline on the Gset benchmark. This result validates the effectiveness of our integrated architectural approach.

Discussion

We developed an FPGA-based sparse Ising machine design that achieves a state-of-the-art solution speed for sparse binary optimization problems of up to 20000 variables on a single FPGA unit. The implementation of the Ising machine on-chip is co-optimized between software layer and hardware design. Compared to previous implementations, this offers a speedup over sparse architectures of 2-3 orders, and a scaleup over dense architectures of 5x. Note that with further optimization of the size of TCOO headers, the FPGA

design proposed in this work is capable of handling larger problem sizes of up to 100000 variables.

The FPGA design is based on main conceptual progress from the point of view of data storage format, data flow, and computing framework. Concretely, we introduced the tiled coordinate format as an efficient format for storing sparse matrices in off-chip memory. The format allows for strongly parallel processing and transmission of data, which has been shown to lead on average to 30% less memory usage compared to the COO format and 22% less memory usage than BCOO. The format also enables better workload balancing between multiple PEs, achieving 95% efficiency on average. Data were quantized to Int8 on a uniform binary scale, leading to 50% less required storage. Intriguingly, the quantized Ising machine was shown to maintain or even increase accuracy over the corresponding full-precision FLOAT32 variants. In combination, a co-optimized FPGA implementation of bSB, called qSBM, was benchmarked against state-of-the-art sparse FPGA implementations and shown to achieve 18-22x speedup on large-scale problems.

A limitation of the current TCOO approach is its use of uniformly-shaped (square) tiles. This leads to suboptimal storage efficiency for matrices with highly irregular, diagonally banded sparsity patterns. The issue is most pronounced at the boundaries of bands, where a uniform tile must encapsulate both dense and sparse regions. Consequently, edge tiles that contain only a few non-zero elements still incur the fixed per-tile metadata overhead. As shown in Supplementary Note 4, this specific edge-tile overhead allows CSR5 (a row-oriented format) to outperform TCOO in a few isolated cases (e.g., graphs G55, G56). For the vast majority of test cases with more uniform or clustered sparsity, TCOO maintains a clear advantage in storage.

From top-level architecture view, the bandwidth between the off-chip DRAM and the FPGA chip is the bottleneck of the computation. Therefore, in future FPGA designs, FPGA board with high-bandwidth memory (HBM) could be used, which typically increases the bandwidth more than four times compared to normal DRAM.

Methods

Ising model, QUBO, and Max-Cut Problems. The subclass of combinatorial optimization problems considered in this work is referred to as quadratic unconstrained binary optimization (QUBO), which optimizes a quadratic polynomial with N variables $\mathbf{s} \in \{0,1\}^N$ over all 2^N possible variable assignments, as⁴³

$$\min_{\mathbf{s} \in \{0,1\}^N} \mathbf{s}^T Q \mathbf{s} , \quad (5)$$

where $Q \in \mathbb{R}^{N \times N}$ is the matrix that encodes the problem, assumed to be symmetric without loss of generality. Many NP-complete and NP-hard problems have been shown to be formulated as QUBOs^{44,45}.

Note that optimization constraints are usually encoded into penalty terms, as^{46,47}

$$\min_{\mathbf{s} \in \{0,1\}^N} \mathbf{s}^T Q \mathbf{s} + \sum_i \alpha_i P_i(\mathbf{s}) . \quad (6)$$

Here, $P_i(\mathbf{s})$ is a general, often quadratic, function of the optimization variables that yields $P_i(\mathbf{s}) = 0$ whenever $\mathbf{s}$ satisfies the constraint. The coefficients $\{\alpha_i\}_i$ require tuning to ensure convergence to accurate solutions.

QUBOs are related to Ising Hamiltonians via the coordinate transformation $\boldsymbol{\sigma} = 2\mathbf{s} - \mathbf{1}$, where $\mathbf{1}$ is the vector with value 1 in each entry, yielding

$$\min_{\boldsymbol{\sigma} \in \{-1,+1\}^N} H_{\text{Ising}}(\boldsymbol{\sigma}) = \min_{\boldsymbol{\sigma} \in \{-1,+1\}^N} -\boldsymbol{\sigma}^T J \boldsymbol{\sigma} - \mathbf{h} \cdot \boldsymbol{\sigma} . \quad (7)$$

The new variables $\boldsymbol{\sigma}$ are called spins. The matrix J is related to the original problem matrix Q , and the vector $\mathbf{h}$ denotes the external field on each individual spin, sometimes referred to as bias. In the main text, the bias term was absorbed into the matrix J without loss of generality by extending the number of variables by one.

The manuscript benchmarks the performance of the algorithms and hardware implementations on the Max-cut problem, which partitions a graph into two subgraphs such that the number of connections between the subgraphs is maximal. The edge weight between node i and j is denoted as w_{ij} . All nodes in one subgraph have value $+1$, and all nodes in the other subgraph -1 . The optimization problem directly assumes the shape of Eq. (7), as

$$\begin{aligned} & \max_{\sigma \in \{-1, +1\}^N} \frac{1}{2} \sum_{i,j} \frac{1 - \sigma_i \sigma_j}{2} w_{ij} \\ &= \min_{\sigma \in \{-1, +1\}^N} \frac{1}{4} \sum_{i,j} \sigma_i \sigma_j w_{ij} + c, \end{aligned} \quad (8)$$

where c is a constant. As the optimization is unconstrained, the bias term vanishes. The Max-cut problem has been a common testbed for combinatorial optimization and Ising solvers^{8–10,13,40,48}. To benchmark Max-cut, we use the Gset graph collection³². It includes graphs ranging from 800 to 20,000+ nodes, with varying non-zero densities and structures.

Solving Ising models with dynamical systems. Ising machines consist of a network of dynamical systems, which encode the optimization problem from Eq. (7) into the interactions between the systems. By evolving the systems under external control mechanisms, the systems output a state configuration which solves the optimization⁴⁹. Many physical networks have been proposed as Ising machines, such as optical networks^{8,50}, coupled oscillator networks^{49,51}, and annealers⁵².

Typically, physical systems are simulated on digital hardware, which leads to solving the equations of motion, a system of coupled differential equations, for fixed points. The general form of the equations reads

$$\frac{d\mathbf{x}}{dt} = F(\mathbf{x}(t), \mathbf{p}(t)) + \xi \nabla H_{\text{Ising}}(\mathbf{x})(t) \quad (9)$$

and the function $F(\mathbf{x}(t), \mathbf{p}(t))$ denotes the dynamics of the individual systems under the influence of time-dependent external control with parameters $\mathbf{p}(t)$. The gradual increase of $\mathbf{p}(t)$ triggers the emergence of new stable states until the system settles in a configuration that corresponds to a solution of the combinatorial optimizations¹³.

In this work, we focus on ballistic simulated bifurcation (bSB) as one particular type of physics-inspired Ising machine, which has been shown to achieve state-of-the-art accuracy for several benchmark problems⁴⁰. bSB is a momentum method such that the states consist of $2N$ variables given by tuples (x_i, y_i) . The self-evolution term is given by $F(\mathbf{x}, \mathbf{y}, \mathbf{p}(t)) = (a\mathbf{y}, (-a\mathbf{1} + \mathbf{p}(t)) \odot \mathbf{x})$, and the coupling term reads $(0, \xi J \cdot \mathbf{x})$. The product $\odot$ stands for the element-wise multiplication of the vectors. The values a and ξ are tunable parameters of the model.

Applying the symplectic Euler method to the equations of motion leads to the following iterative update rule⁴⁰:

$$\begin{aligned} \mathbf{y}(t + \Delta t) &= \mathbf{y}(t) + \Delta t \cdot [(-a\mathbf{1} + \mathbf{p}(t)) \odot \mathbf{x} + \xi J \cdot \mathbf{x}(t)], \\ \mathbf{x}(t + \Delta t) &= \mathbf{x}(t) + \Delta t \cdot \mathbf{y}(t + \Delta t). \end{aligned} \quad (10)$$

After each iteration, the variables are bounded at $\text{sign}(\mathbf{x})$ and the momenta set to $y_i = 0$ if $|x_i| > 1$. A detailed overview of algorithmic implementations of bSB and quantized bSB is given in Supplementary Note 1.

Embedding Ising dynamics into FPGA. Ising dynamics can be divided into SpMV and TE (see Eq. 10). Hence, we use a scatter-and-gather paradigm^{53,54} to embed the dynamics in the FPGA.

In the scatter phase, SpMV $J \cdot \mathbf{x}$ is computed using parallel LUT arrays configured as PEs. Each PE handles a subset of non-zero entries J_{ij} , performing integer arithmetic for energy efficiency:

$$(J \cdot \mathbf{x})_i = \sum_{j \in N(i)} J_{ij} x_j ,$$

where $N(i)$ denotes the nearest neighbors of spin i . Since $\mathbf{x}$ is quantized and J is an integer matrix, we can use LUT to implement integer operations as follows. Consider two k -bit integers A and B . Then their multiplication reads

$$A \cdot B = \sum_{i=0}^{k-1} (A \cdot b_i) 2^i ,$$

where b_i is the i -th bit of B . In LUT implementation, each term $A \cdot b_i$ is a bitwise AND, computed by one or more k -input LUT configured as a truth table.

In the gather phase, TE, encompassing all operations outside of SpMV (typically fixed-point operations), is mapped to digital signal processing (DSP) blocks.

To bridge the two computing modules, a ping-pong Random Access Memory (RAM) stores the SpMV outcomes from PEs for subsequent GE processing, enabling simultaneous execution.

SpMV optimization. The execution of SpMV has some intrinsic hazards. Data hazards emerge when multiple threads violate the execution order constraints for shared result vector elements r_k . Intra-stage hazards occur when concurrent threads read-modify-write the same r_k within a single pipeline stage S_j , leading to unsynchronized updates. For instance, if two threads simultaneously read a vector element r_k for accumulation, each thread updates the vector element individually as $r_k \rightarrow r_k + \Delta_1$ and $r_k \rightarrow r_k + \Delta_2$, while the correct result requires $r_k \leftarrow r_k + \Delta_1 + \Delta_2$. Inter-stage read-after-write (RAW) hazards arise when a stage S_{j+1} prematurely fetches r_k before the pending write from stage S_j . Structural hazards arise from hardware resource contention. When N threads attempt to write results via a BRAM with only two ports ($N \gg 2$), the system must resolve conflicts through stalling or rescheduling, directly capping throughput at

$$\text{Throughput}_{\max} = \frac{2 \cdot f_{\text{BRAM}}}{\text{cycles/write}} .$$

To resolve this, the PE employs a hazard detection unit which controls the later part of SpMV logic. It decides whether a single multiplied result in row x needs to be merged with other results in the same pipeline stage or forwarded to other pipeline stages, avoiding the wrong SpMV result.

Scalability. Prior SA-based Ising machines typically support 3-5 bit matrix non-zeros weights⁵⁵, and the dense SBM design **K2000** is limited to 2 bit. The current hardware design natively supports 4-bit non-zeros weights and activations to maintain algorithmic generality and processing speed, as increasing the bit-width further would lead to diminished processing capability and increase the resource utilization (particularly DSP blocks and LUT). Note that the architecture itself is fundamentally scalable to higher precisions (e.g., 8-bit integer or 16-bit floating-point) without structural modification, as the core dataflow and processing elements are parameterized by bit-width.

Data Availability

The plots data generated in this study have been deposited in the Zenodo database under accession code [10.5281/zenodo.20249235](https://doi.org/10.5281/zenodo.20249235).

Code Availability

The code that supports the plots within this paper and other findings of this study is available from the corresponding authors upon request. It is not deposited in a public repository because it contains proprietary algorithms that are part of a pending patent application and are subject to ongoing commercial development. Requests for access should be directed to the corresponding authors (ebler.dan@gmail.com, wei.zhang@ust.hk, riosun@gmail.com) and will be responded to within four weeks of receipt.

References

- [1] Wolsey, L. A. & Nemhauser, G. L. *Integer and combinatorial optimization* (John Wiley & Sons, 1999).
- [2] Ausiello, G. *et al.* *Complexity and approximation: Combinatorial optimization problems and their approximability properties* (Springer Science & Business Media, 2012).
- [3] Vesselinova, N., Steinert, R., Perez-Ramirez, D. F. & Boman, M. Learning combinatorial optimization on graphs: A survey with applications to networking. *IEEE Access* **8**, 120388–120416 (2020).
- [4] Naseri, G. & Koffas, M. A. Application of combinatorial optimization strategies in synthetic biology. *Nature communications* **11**, 2446 (2020).
- [5] Böhm, F., Alonso-Urquijo, D., Verschaffelt, G. & Van der Sande, G. Noise-injected analog ising machines enable ultrafast statistical sampling and machine learning. *Nature Communications* **13**, 5847 (2022).
- [6] Kaseb, Z., Möller, M., Vergara, P. P. & Palensky, P. Power flow analysis using quantum and digital annealers: a discrete combinatorial optimization approach. *Scientific Reports* **14**, 23216 (2024).
- [7] Bian, Z., Chudak, F., Macready, W. G. & Rose, G. The ising model: teaching an old problem new tricks. *D-wave systems* **2**, 1–32 (2010).
- [8] Marandi, A., Wang, Z., Takata, K., Byer, R. L. & Yamamoto, Y. Network of time-multiplexed optical parametric oscillators as a coherent ising machine. *Nature Photonics* **8**, 937–942 (2014).
- [9] Inagaki, T. *et al.* A coherent ising machine for 2000-node optimization problems. *Science* **354**, 603–606 (2016).
- [10] Goto, H., Tatsumura, K. & Dixon, A. R. Combinatorial optimization by simulating adiabatic bifurcations in nonlinear hamiltonian systems. *Science advances* **5**, eaav2372 (2019).
- [11] Böhm, F., Verschaffelt, G. & Van der Sande, G. A poor man’s coherent ising machine based on optoelectronic feedback systems for solving optimization problems. *Nature communications* **10**, 3538 (2019).
- [12] Mohseni, N., McMahon, P. L. & Byrnes, T. Ising machines as hardware solvers of combinatorial optimization problems. *Nature Reviews Physics* **4**, 363–379 (2022).
- [13] Wang, J., Ebler, D., Wong, K. M., Hui, D. S. W. & Sun, J. Bifurcation behaviors shape how continuous physical dynamics solves discrete ising optimization. *Nature Communications* **14**, 2510 (2023).
- [14] Bybee, C. *et al.* Efficient optimization with higher-order ising machines. *Nature Communications* **14**, 6033 (2023).
- [15] González, V. H., Litvinenko, A., Kumar, A., Khymyn, R. & Åkerman, J. Spintronic devices as next-generation computation accelerators. *Current Opinion in Solid State and Materials Science* **31**, 101173 (2024).
- [16] Zhang, Y. *et al.* Oscillator-network-based ising machine. *micromachines* **13**, 1016 (2022).
- [17] Wang, T. & Roychowdhury, J. Oim: Oscillator-based ising machines for solving combinatorial optimisation problems. In *International Conference on Unconventional Computation and Natural Computation*, 232–256 (Springer, 2019).

- [18] Vadlamani, S. K., Xiao, T. P. & Yablonovitch, E. Physics successfully implements lagrange multiplier optimization. *Proceedings of the National Academy of Sciences* **117**, 26639–26650 (2020).
- [19] Shen, Z.-S. *et al.* Free-energy machine for combinatorial optimization. *Nature Computational Science* 1–11 (2025).
- [20] Tatsumura, K., Dixon, A. R. & Goto, H. Fpga-based simulated bifurcation machine. In *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, 59–66 (IEEE, 2019).
- [21] Zou, Y. & Lin, M. Massively simulating adiabatic bifurcations with fpga to solve combinatorial optimization. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 65–75 (2020).
- [22] Aadit, N. A. *et al.* Massively parallel probabilistic computing with sparse ising machines. *Nature Electronics* **5**, 460–468 (2022).
- [23] Kashimata, T., Yamasaki, M., Hidaka, R. & Tatsumura, K. Efficient and scalable architecture for multiple-chip implementation of simulated bifurcation machines. *arXiv preprint arXiv:2311.17370* (2023).
- [24] Yan, S., Li, C., Zhang, Y. & Zhou, H. yasmv: Yet another spmv framework on gpus. *Acm Sigplan Notices* **49**, 107–118 (2014).
- [25] Leleu, T., Yamamoto, Y., McMahon, P. L. & Aihara, K. Destabilization of local minima in analog spin systems by correction of amplitude heterogeneity. *Physical review letters* **122**, 040607 (2019).
- [26] Cook, C., Zhao, H., Sato, T., Hiromoto, M. & Tan, S. X.-D. Gpu-based ising computing for solving max-cut combinatorial optimization problems. *Integration* **69**, 335–344 (2019).
- [27] Nakayama, H., Koyama, J., Yoneoka, N. & Miyazawa, T. Description: Third generation digital annealer technology. *Fujitsu Limited: Tokyo, Japan* (2021).
- [28] Willsch, D. *et al.* Benchmarking advantage and d-wave 2000q quantum annealers with exact cover problems. *Quantum Information Processing* **21**, 141 (2022).
- [29] Carugno, C., Ferrari Dacrema, M. & Cremonesi, P. Evaluating the job shop scheduling problem on a d-wave quantum annealer. *Scientific Reports* **12**, 6539 (2022).
- [30] Quinton, F. A., Myhr, P. A. S., Barani, M., Crespo del Granado, P. & Zhang, H. Quantum annealing applications, challenges and limitations for optimisation problems compared to classical solvers. *Scientific Reports* **15**, 12733 (2025).
- [31] Saad, Y. Sparskit: a basic tool kit for sparse matrix computations, version 2. <http://www.cs.umn.edu/saad/software/SPARSKIT/sparskit.html> (1994).
- [32] University, S. Gset. <https://web.stanford.edu/~yyye/yyye/Gset/>.
- [33] Liu, W. & Vinter, B. Csr5: An efficient storage format for cross-platform sparse matrix-vector multiplication. In *Proceedings of the 29th ACM on International Conference on Supercomputing*, 339–350 (2015).
- [34] Sadi, F. *et al.* Efficient spmv operation for large and highly sparse matrices using scalable multi-way merge parallelization. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 347–358 (2019).
- [35] Jacob, B. *et al.* Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2704–2713 (2018).

- [36] Wu, H., Judd, P., Zhang, X., Isaev, M. & Micikevicius, P. Integer quantization for deep learning inference: Principles and empirical evaluation. *arXiv preprint arXiv:2004.09602* (2020).
- [37] Gholami, A. *et al.* A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*, 291–326 (Chapman and Hall/CRC, 2022).
- [38] Nagel, M. *et al.* A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295* (2021).
- [39] Zhang, T. & Han, J. Quantized simulated bifurcation for the ising model. In *2023 IEEE 23rd International Conference on Nanotechnology (NANO)*, 715–720 (IEEE, 2023).
- [40] Goto, H. *et al.* High-performance combinatorial optimization based on classical mechanics. *Science Advances* **7**, eabe7953 (2021).
- [41] Zick, K. M. Performance report of heuristic algorithm that cracked the largest gset ising problems (g81 cut= 14060). *arXiv preprint arXiv:2505.18508* (2025).
- [42] Furini, F. *et al.* Qplib: a library of quadratic programming instances. *Mathematical Programming Computation* **11** (2019).
- [43] Kochenberger, G. *et al.* The unconstrained binary quadratic programming problem: a survey. *Journal of combinatorial optimization* **28**, 58–81 (2014).
- [44] Barahona, F. On the computational complexity of ising spin glass models. *Journal of Physics A: Mathematical and General* **15**, 3241 (1982).
- [45] Karp, R. M. Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*, 219–241 (Springer, 2009).
- [46] Liu, J. & Moraglio, A. A framework for automatically setting multiple penalty weights in quadratic unconstrained binary optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 575–578 (2025).
- [47] Ayodele, M. Penalty weights in qubo formulations: Permutation problems. In *European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar)*, 159–174 (Springer, 2022).
- [48] Tiunov, E. S., Ulanov, A. E. & Lvovsky, A. Annealing by simulating the coherent ising machine. *Optics express* **27**, 10288–10295 (2019).
- [49] Goto, H. Bifurcation-based adiabatic quantum computation with a nonlinear oscillator network. *Scientific reports* **6**, 21686 (2016).
- [50] Yamamoto, Y. *et al.* Coherent ising machines—optical neural networks operating at the quantum limit. *npj Quantum Information* **3**, 49 (2017).
- [51] Chou, J., Bramhavar, S., Ghosh, S. & Herzog, W. Analog coupled oscillator based weighted ising machine. *Scientific reports* **9**, 14786 (2019).
- [52] Kadowaki, T. & Nishimori, H. Quantum annealing in the transverse ising model. *Physical Review E* **58**, 5355 (1998).
- [53] Kyrola, A., Blelloch, G. & Guestrin, C. {GraphChi}:{Large-Scale} graph computation on just a {PC}. In *10th USENIX symposium on operating systems design and implementation (OSDI 12)*, 31–46 (2012).
- [54] Zhu, X., Han, W. & Chen, W. {GridGraph}:{Large-Scale} graph processing on a single machine using 2-level hierarchical partitioning. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, 375–386 (2015).
- [55] Takemoto, T. *et al.* 4.6 a 144kb annealing system composed of 9×16 kb annealing processor chips with scalable chip-to-chip connections for large-scale combinatorial optimization problems. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, 64–66 (IEEE, 2021).

Acknowledgements

The authors thank Chaofang Ma for insightful discussions and suggestions on the development of the SpMV-related components of this work. We also thank Zhongchun Zhou for providing the Ising simulation code.

Funding Statement

This work was supported by the Research Grants Council of Hong Kong under General Research Fund (GRF) grant number 16218324 and Collaborative Research Fund (CRF) grant number C5032-23G.

Author Contributions Statement

J.S. and W.Z. conceived the overall project and directed the research. B.Y. and X.S. contributed to the project conceptualization. J.W. provided the Ising simulation code. B.Y. and X.S. designed the quantized Ising algorithm, hardware architecture, and FPGA RTL, and wrote the initial draft of the manuscript. D.E. and F.Z. contributed to the theoretical interpretation of the Ising simulation. D.E., W.Z., and J.S. led the critical manuscript revisions. All authors discussed the results and contributed to the final version of the manuscript. All authors reviewed and approved the submitted manuscript.

Competing Interests Statement

The authors declare no competing interests.